

M14 - Kafka Streaming

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

Concept before command

Every lesson explains the mental model and correctness/performance evidence before asking you to run the project.

Contents

- DE14-01** - Batch vs streaming
- DE14-02** - Events, brokers and topics
- DE14-03** - Producers and message keys
- DE14-04** - Consumers and safe processing
- DE14-05** - Partitions and ordering
- DE14-06** - Offsets, group position and replay
- DE14-07** - Consumer groups and scaling
- DE14-08** - Delivery semantics and idempotency
- DE14-09** - Serialization, schemas and evolution
- DE14-10** - Python producer and consumer workflow
- DE14-11** - Spark Structured Streaming + Kafka
- DE14-12** - Lag, failures and streaming operations

DE14-01 - Batch vs streaming

What you will be able to do

Distinguish bounded batch datasets from unbounded event streams and choose processing model by latency/change requirements.

Why this matters

Streaming is not "batch but faster". The source is logically unbounded and state/progress/failure handling is continuous.

Picture it this way

Batch is receiving a closed box of yesterday's receipts; streaming is receipts arriving one by one on a conveyor belt that does not end.

Start from zero

- Batch has a known/bounded input for one run/window.
- Streaming consumes continuously from a position in an event log/source.
- Both still process finite records in tasks/micro-batches internally; streaming describes unbounded semantics/latency.
- Choose streaming when business value requires low latency/event-driven behavior, not for resume buzzword.
- Streaming requires progress/offset/checkpoint and duplicate/replay reasoning.
- Historical recomputation/backfill is still needed in streaming systems.

Teacher demonstration

```
python M14_G01_BATCH_VS_STREAM.py
# Compare BATCH_ROWS and STREAM_EVENTS outputs/mental model.
```

Read the example like English

- The lab is pure Python and works without Kafka runtime.
- The same records can be processed in one bounded list or as sequential events; operational semantics differ.

Expected check

batch	stream
finite input/run	unbounded source position
schedule/window	continuous/event arrivals
rerun batch	replay offsets/events

Common beginner traps

- Calling streaming real-time automatically.
- Using streaming when daily batch meets SLA.
- Ignoring replay/backfill because source is live.

Now you do a similar task

For three business cases (daily finance report, fraud alert, hourly inventory), choose batch/stream/hybrid and defend latency/cost/complexity.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

What makes a stream unbounded even though each individual message is finite?

Answer in your own words before continuing.

DE14-02 - Events, brokers and topics

What you will be able to do

Explain event, broker, topic and Kafka log model before writing producer/consumer code.

Why this matters

If topic/partition/offset are unclear, consumer groups and replay feel like magic.

Picture it this way

Kafka is a durable post office: producers append letters to named topic logs; consumers read them at remembered positions.

Start from zero

- An **event/message** has key/value plus metadata/headers/timestamp.
- A Kafka broker stores topic partitions and serves producer/consumer requests.
- A **topic** is a named logical stream split into partitions.
- Partitions are append-only ordered logs with offsets.
- Retention is based on Kafka configuration/time/size/compaction policy, not on whether a consumer read a message.
- Kafka is not a job scheduler or database table in the same sense as PostgreSQL.

Teacher demonstration

```
topic: orders
partition 0: [offset0][offset1][offset2]...
partition 1: [offset0][offset1]...
producer -> broker/topic partitions -> consumer
```

Read the example like English

- Offsets are per partition; offset 5 in partition 0 differs from offset 5 in partition 1.
- Consumers can reread retained records by moving position.

Expected check

term	meaning
broker	Kafka server/storage node
topic	named stream
partition	ordered log shard
offset	position within one partition

Common beginner traps

- Thinking one global offset for whole topic.
- Assuming consumed messages disappear.
- Calling topic a relational table.

Now you do a similar task

Run the pure partition/order lab and draw a two-partition topic with offsets. Explain which order guarantees exist.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why must an offset always be interpreted together with a partition?

Answer in your own words before continuing.

DE14-03 - Producers and message keys

What you will be able to do

Produce messages deliberately with key, value serialization and delivery acknowledgement/error handling.

Why this matters

Producer key choice controls partition affinity/order for related events; ignoring delivery errors can lose messages silently.

Picture it this way

The key is the routing label on a parcel: all events for one order can be sent to the same conveyor lane to preserve their relative order.

Start from zero

- Producer serializes key/value bytes and sends to topic partition.
- If no partition specified, partitioner commonly uses key or balancing logic depending client/config.
- Stable key such as order_id can keep same-key events in one partition.
- A hot key can create partition skew.
- Asynchronous produce requires polling/flush/delivery callbacks/ack handling before process exits.
- Do not log sensitive message payloads/tokens casually.

Teacher demonstration

```
from confluent_kafka import Producer
p=Producer({"bootstrap.servers":"localhost:9092"})
p.produce("orders", key="01001", value='{"status":"PAID"}')
p.flush()
```

Read the example like English

- key 01001 is used by partitioner rather than forcing a partition number in this simple example.
- flush waits for outstanding delivery attempts before process exits; production code should inspect delivery callbacks/errors.

Expected check

decision	reason
key=order_id	same-order ordering/partition affinity
no key	distribution but no same-entity affinity guarantee
flush/callback	delivery evidence

Common beginner traps

- Producer script exits before outstanding messages handled.
- Random key per update destroys same-entity ordering.
- One hot constant key puts all messages in one partition.

Now you do a similar task

Inspect/run producer lab if Kafka runtime is available; otherwise offline-validate. Explain key choice for order updates and one skew risk.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why can using `customer_id` as key be better or worse than `order_id` depending the workload?

Answer in your own words before continuing.

DE14-04 - Consumers and safe processing

What you will be able to do

Consume messages with explicit group/commit behavior and ensure durable side effects happen before acknowledging progress.

Why this matters

A consumer can crash between processing and committing offset. Correct ordering determines whether data is lost or replayed.

Picture it this way

Do the work, get a receipt from the target, then move the bookmark. Moving bookmark first can skip unfinished letters.

Start from zero

- Consumer subscribes/assigns partitions and polls records.
- Group ID identifies progress/coordination for a consumer group.
- Offset commit records next/resume position for the group according to client semantics.
- For at-least-once processing, write durable idempotent target first, then commit offset.
- If crash occurs after target write before commit, message may replay; target must handle duplicate.
- If commit occurs before target write and crash follows, data can be lost/skipped.

Teacher demonstration

```
# Safe conceptual order
msg = consumer.poll(...)
validate(msg)
write_target_idempotently(msg)
consumer.commit(message=msg, asynchronous=False)
```

Read the example like English

- Target write precedes offset commit.
- Replay window remains possible, so idempotency is necessary.
- Synchronous commit is simpler to reason about in a beginner lab; throughput tradeoffs exist.

Expected check

crash point	effect
before target	message replay
after target before commit	message replay -> duplicate risk
after commit	next message resumes
commit before target then crash	loss/skip risk

Common beginner traps

- auto-committing progress before durable processing without understanding config.
- Treating commit as database transaction with target.
- No poison-message/error policy.

Now you do a similar task

Inspect consumer lab config/order. Write a failure timeline for three crash points and target behavior.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why does committing an offset after the target write usually trade possible duplicates for protection against loss?

Answer in your own words before continuing.

DE14-05 - Partitions and ordering

What you will be able to do

Explain partition ordering and key-based affinity without claiming global topic ordering.

Why this matters

Kafka's scalability comes from partitions; ordering is normally guaranteed within a partition, not across partitions.

Picture it this way

Each checkout lane preserves its own customer sequence, but there is no single ordering between two different lanes.

Start from zero

- Each partition has its own monotonically increasing offsets/order.
- Same key normally maps to same partition while partition count/partitioner configuration remains compatible.
- Different keys/partitions have no inherent total order.
- Adding partitions can change key-to-partition mapping for future messages, affecting assumptions.
- A single partition gives simple total ordering but limits parallel consumer scaling.
- Design key/order requirements from entity semantics.

Teacher demonstration

```
python M14_G02_PARTITION_ORDER.py  
# Inspect PARTITION_OFFSETS and ORDERED_WITHIN_PARTITION.
```

Read the example like English

- Pure Python lab demonstrates per-partition order.
- The relative order of partition 0 offset 5 and partition 1 offset 8 is not defined by Kafka log order alone.

Expected check

ordering	guarantee
within one partition	yes by offset
same key -> same partition	normally under stable partitioner/partition count
across partitions	no global order

Common beginner traps

- Sorting by offset across partitions.
- One partition for everything solely to simplify reasoning.
- Changing partition count without considering keyed ordering.

Now you do a similar task

Create events for two order IDs across two partitions and explain every ordering statement you can/cannot make.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Can you say partition0 offset10 happened before partition1 offset4? Why not?

Answer in your own words before continuing.

DE14-06 - Offsets, group position and replay

What you will be able to do

Use offsets as replay/progress positions and distinguish current consumer position from committed group offset.

Why this matters

Replay and recovery depend on exactly which partition offsets were processed and committed.

Picture it this way

Offset is a bookmark inside one partition log; the group's committed bookmark may lag behind what a running consumer has already fetched.

Start from zero

- Offsets identify records within a partition.
- Consumer position is where the current instance is reading; committed offset is durable group progress used after restart/rebalance.
- `auto.offset.reset` controls start behavior when no valid committed offset exists (earliest/latest etc.), not a magic replay button for existing commits.
- Replay can assign/seek to earlier offsets or use a new group depending use case/tool.
- Replaying retained events must not duplicate durable business effects.
- Record replay range/reason/run identity for audit.

Teacher demonstration

```
# Conceptual explicit replay
TopicPartition("orders", 0, 5)
# Start partition 0 at offset 5, then process using idempotent target.
```

Read the example like English

- Partition + offset together define the replay start.
- Changing the group or seeking can reread old retained records.
- Downstream must distinguish/reapply safely.

Expected check

state	meaning
consumer position	current in-process next read
committed offset	durable group checkpoint
earliest/latest	fallback when no usable commit
seek/assign	explicit replay position

Common beginner traps

- Setting `auto.offset.reset=earliest` and assuming it ignores existing commits.
- Deleting group offsets casually in production.
- Replay without idempotent sink.

Now you do a similar task

Run offset/replay pure/offline lab. Explain how you would replay partition 1 offsets 10-20 without disturbing normal group progress.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

What is the difference between a committed offset and the current consumer position?

Answer in your own words before continuing.

DE14-07 - Consumer groups and scaling

What you will be able to do

Scale consumers with consumer groups while respecting one-active-consumer-per-partition assignment within a group.

Why this matters

Partitions define maximum parallelism for one consumer group at a moment. Extra consumers beyond partition count sit idle.

Picture it this way

A topic partition is a checkout lane; within one team/group, one worker owns a lane at a time so two workers do not both process the same lane simultaneously.

Start from zero

- Consumers with same group.id cooperate; Kafka assigns partitions among them.
- Within a group, one partition is assigned to at most one consumer at a time.
- Maximum active parallel consumers approximately partition count.
- Different consumer groups read the same topic independently with separate offsets (e.g., fraud group and analytics group).
- Membership changes trigger rebalances/assignment changes; processing/commit design must tolerate them.
- Scaling partitions has ordering/key-distribution implications.

Teacher demonstration

```
# Example: 3 partitions
Group analytics:
  consumer A -> p0,p1
  consumer B -> p2
  consumer C -> idle (or assignment varies)
Group fraud:
  consumer X -> p0,p1,p2 independently
```

Read the example like English

- Two groups each receive the stream independently.
- Within analytics group, p2 is not simultaneously processed by A and B.

Expected check

partitions	consumers in same group
3	1 -> handles 3
3	2 -> split
3	3 -> ~1 each
3	5 -> ~2 idle

Common beginner traps

- Adding consumers without adding partitions and expecting unlimited throughput.
- Giving unrelated applications same group ID accidentally, causing them to split messages.
- Assuming rebalance preserves in-memory state automatically.

Now you do a similar task

Run consumer-groups lab/offline validation and draw assignments for 4 partitions with 1,2,4,6 consumers.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why do two independent applications usually need different group IDs if both must see every event?

Answer in your own words before continuing.

DE14-08 - Delivery semantics and idempotency

What you will be able to do

Reason about at-most-once/at-least-once/exactly-once claims and build idempotent sinks to make replay safe.

Why this matters

Delivery semantics are end-to-end. Kafka/client settings alone do not guarantee your database/API side effect occurs exactly once.

Picture it this way

If a courier may redeliver a parcel after uncertain acknowledgement, the receiver needs a unique parcel ID to avoid charging twice.

Start from zero

- At-most-once may commit/progress before processing -> possible loss, fewer duplicates.
- At-least-once processes then commits -> replay/duplicates possible, less loss.
- Exactly-once requires coordinated transactional semantics across source/processing/sink scope; be precise about what system boundary supports it.
- Idempotent sink uses event ID/business key/version/upsert/dedup table so duplicate event has same effect.
- Store processed event identity only atomically with business effect where possible.
- Poison messages need retry/dead-letter/quarantine policy.

Teacher demonstration

```
# Current-state sink idea
INSERT ... ON CONFLICT (order_id)
DO UPDATE SET status = EXCLUDED.status
WHERE target.event_version < EXCLUDED.event_version;
```

Read the example like English

- Key+version makes replay/state events safe for current-state semantics.
- For additive facts/payments, idempotency may require unique event_id rather than order_id.

Expected check

semantic	main risk
at-most-once	loss
at-least-once	duplicates
exactly-once claim	must define boundary
idempotent sink	duplicate delivery -> same durable effect

Common beginner traps

- Saying Kafka is exactly once everywhere by default.
- Blind append sink under at-least-once consumer.
- Using mutable payload hash as event identity without contract.

Now you do a similar task

Run delivery-semantics pure lab and design an idempotent database write for a payment_event_id versus order current status.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why might order_id be the correct idempotency key for current status but wrong for multiple payment events?

Answer in your own words before continuing.

DE14-09 - Serialization, schemas and evolution

What you will be able to do

Serialize events with an explicit schema/version and handle compatible/incompatible evolution/quarantine.

Why this matters

Streaming producers/consumers deploy independently; an uncoordinated field/type change can break continuously running consumers.

Picture it this way

Event schema is the shared form contract passed on the conveyor belt; producer version changes must keep existing readers in mind.

Start from zero

- JSON is human-readable but still needs a schema/contract for required fields/types/meaning.
- Binary formats such as Avro/Protobuf commonly pair with schema registries in production, but core lesson is compatibility/versioning.
- Adding optional field with default can be compatible; removing/renaming required field or changing type/meaning can be breaking.
- Include event_type/schema_version where useful and validate before processing.
- Quarantine bad/unknown schema events with key/offset metadata instead of blocking forever without visibility.
- Do not deserialize untrusted arbitrary code objects (pickle) from event streams.

Teacher demonstration

```
required={"event_id", "order_id", "event_type", "event_time"}
obj=json.loads(message_value)
missing=required-set(obj)
if missing:
    quarantine(topic,partition,offset,reason=f"missing {missing}")
```

Read the example like English

- The consumer validates contract before business side effects.
- Quarantine evidence includes stream position so event can be repaired/replayed.

Expected check

change	compatibility tendency
add optional field	usually compatible
remove required field	breaking
numeric -> string meaning	breaking
new event_type	consumer policy required

Common beginner traps

- Parsing JSON successfully and assuming schema is valid.
- Using pickle from untrusted topic.
- Consumer crashes/retries poison event forever with no quarantine.

Now you do a similar task

Run serialization/schema lab. Feed one valid, one compatible-extra-field and one missing-required event; save accepted/rejected counts/reasons.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why should quarantine include topic/partition/offset, not only the payload?

Answer in your own words before continuing.

DE14-10 - Python producer and consumer workflow

What you will be able to do

Run a Python Kafka producer/consumer workflow with explicit key/group/commit and observable message metadata.

Why this matters

The hands-on goal is to connect the conceptual log model to real client behavior without hiding offsets/keys/errors.

Picture it this way

You send labeled parcels, watch which lane/offset they land in, consume them with a named team and commit progress after safe handling.

Start from zero

- Start/verify Kafka training broker from Compose before rewriting client code.
- Producer config needs bootstrap.servers; delivery callback/flush proves send attempts.
- Consumer config needs bootstrap.servers, group.id and offset-reset/auto-commit policy.
- Inspect msg.topic(), partition(), offset(), key(), value() and error.
- Commit only after safe processing for the chosen semantics.
- Use fake training data/credentials only; local broker is for learning, not production security model.

Teacher demonstration

```
# Terminal A
# docker compose up -d
# Terminal B
python M14_G03_PRODUCER.py
# Terminal C
python M14_G04_CONSUMER.py
```

Read the example like English

- Producer/consumer scripts are supplied and offline syntax validated.
- If Docker/Kafka runtime is unavailable, do not mark live competency complete based only on static validation.
- Save actual partition/offset messages when running live.

Expected check

runtime proof	evidence
broker	compose service healthy/log
producer	delivery success/key
consumer	group + partition/offset
commit	after processing

Common beginner traps

- Changing bootstrap server randomly before checking broker/port.
- No delivery callback/flush.
- Marking live Kafka complete from offline parser only.

Now you do a similar task

Run the live workflow when Docker works. Send several keyed order events, record received key/partition/offset and commit behavior.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Which three message metadata fields are most useful when diagnosing a replay/ordering issue?

Answer in your own words before continuing.

DE14-11 - Spark Structured Streaming + Kafka

What you will be able to do

Connect Spark Structured Streaming to Kafka and understand streaming checkpoints/output semantics without collecting the stream to driver.

Why this matters

Distributed streaming combines Kafka offsets with Spark query progress/state and a sink. Checkpoint design determines restart/recovery.

Picture it this way

Kafka supplies the continuous conveyor; Spark groups/processes events in distributed batches while a checkpoint notebook records query progress/state.

Start from zero

- `readStream.format("kafka")` creates a streaming DataFrame with key/value/topic/partition/offset/timestamp columns.
- Cast/parse value bytes using schema; keep stream positions for lineage/debug.
- Transformations remain DataFrame-style; unsupported operations depend on streaming mode/state.
- `writeStream` needs a sink and `checkpointLocation` for recoverable progress/state.
- Checkpoint directory is part of query identity/state; do not reuse/delete casually between unrelated queries.
- End-to-end sink semantics depend on sink/output mode/idempotency, not only source offsets.

Teacher demonstration

```
events=(spark.readStream
  .format("kafka")
  .option("kafka.bootstrap.servers","localhost:9092")
  .option("subscribe","orders")
  .load())
query=(parsed.writeStream
  .option("checkpointLocation","checkpoints/orders_v1")
  .start(...))
```

Read the example like English

- Kafka source columns include partition/offset metadata.
- Checkpoint stores streaming query progress/state so restart can resume consistently.
- The sink still needs its own correctness/idempotency semantics.

Expected check

component	state
Kafka group/source	topic partitions/offsets
Spark query	checkpoint progress/state
sink	durable output/idempotency

Common beginner traps

- Deleting checkpoint to "fix" a stuck query without understanding replay.
- toPandas/collect on unbounded streaming DF.
- Same checkpoint path for different query topology.

Now you do a similar task

Inspect/offline-validate Spark Kafka script. If runtime is available, run a bounded training stream and save checkpoint/output evidence. Explain restart behavior.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why is a Spark streaming checkpoint not the same thing as a backup of Kafka messages?

Answer in your own words before continuing.

DE14-12 - Lag, failures and streaming operations

What you will be able to do

Monitor consumer lag, hot partitions, failures/rebalances and operate replay safely using observable positions.

Why this matters

Streaming health is about keeping up continuously. A running consumer with growing lag is not healthy.

Picture it this way

Lag is the queue of unread letters between broker's latest position and the consumer group's committed position.

Start from zero

- Consumer lag per partition is roughly log-end offset minus committed group offset (with exact tooling semantics).
- Monitor total lag and hottest partition; average can hide one stuck/skewed partition.
- Growing lag can come from slow processing, sink bottleneck, skew/hot key, errors/retries or insufficient partitions/consumers.
- Rebalances can pause processing and move partition ownership; design commits/state safely.
- Alert on sustained lag/error/restart patterns tied to SLA, not one transient spike.
- Replay/offset reset is an operational change: record range/reason and ensure idempotent sink.
- Dead-letter/quarantine poison events so one deterministic bad message does not stall progress forever.

Teacher demonstration

```
python M14_G09_LAG_FAILURE.py
# Pure lab computes TOTAL_LAG and HOTTEST partition from synthetic offsets.
```

Read the example like English

- The pure lab teaches lag math without needing a broker.
- In live Kafka use admin/consumer tooling/metrics for real group end/committed offsets.
- Hottest partition can reveal key skew.

Expected check

signal	meaning
total lag	overall backlog
max partition lag	hot/stuck shard
consumer errors/restarts	failure instability
rebalance frequency	membership/processing issue
DLQ/quarantine count	poison input trend

Common beginner traps

- Monitoring only consumer process uptime.
- Resetting offsets as first fix for lag.
- Adding consumers beyond partition count.

Now you do a similar task

Run lag/failure lab and diagnose three scenarios: all partitions evenly behind, one hot partition, repeated poison message. Give first evidence/remedy for each.

How to prove it

- Save actual command/code/config.
- Save observed plan/output/state/control totals.
- State grain/partition/key/checkpoint assumption.
- Repair disagreement before move-on.

STOP - check your understanding

Why can total lag look acceptable while one partition is still violating an SLA?

Answer in your own words before continuing.